

# Ricci Pruning: Curvature-Guided KV Cache Compression for Long-Context LLMs

Anonymous Authors<sup>1</sup>

## Abstract

Long-context language models are increasingly deployed with 100K-token windows, but autoregressive attention over such histories is dominated by key-value (KV) memory and compute. Existing KV compression and token merging rely on attention magnitudes or feature similarity and lack a principled view of redundancy. We instead treat the token sequence inside a Transformer as a discrete manifold with a Markov kernel and use discrete Ricci curvature to drive compression. *Ricci Pruning* identifies high-curvature redundant regions and merges them, while preserving a low-curvature backbone that mediates long-range dependencies. *Ricci Flow Token Merging* adds a discrete Ricci flow across layers that contracts flat regions and stabilizes merging. Under mild Lipschitz assumptions, we bound the attention error induced by merging curvature-controlled patches, giving an operational notion of near-lossless compression. On long-context benchmarks, our methods achieve up to 80% KV reduction and 2–4 $\times$  speedup with < 1% performance drop, outperforming attention- and similarity-based baselines at matched compression ratios.

## 1. Introduction

Scaling context length has become a central axis of progress for large language models (LLMs). Architectures such as Longformer, BigBird, and recurrent memory transformers have extended effective context windows from 4K to over 1M tokens by introducing sparse attention patterns or explicit memory states (??). At the same time, empirical studies show that current models often fail to exploit long contexts reliably and tend to be “lost in the mid-

dle” (?).

Even with architectural sparsity, *autoregressive* inference over long contexts remains bottlenecked by the size of the key-value (KV) cache and the cost of attending over it. Each generated token attends to all previous tokens, and KV cache memory grows linearly with context length and number of layers. For 100K+ tokens, this dominates GPU memory and latency.

A wave of recent work compresses the KV cache or prunes tokens on the fly. Methods like FastGen, MiniCache, SnapKV, KVQuant, L2-based eviction, and related approaches use attention statistics, depth-wise distillation, snapshot selection, or quantization to reduce memory and computation while trying to preserve quality (????). In parallel, token merging and pruning techniques from the vision community (e.g., ToMe, DynamicViT, dynamic token pruning) merge or drop tokens based on local similarity or learned importance scores (????).

Despite impressive engineering, these methods remain largely *heuristic*: they answer *which* tokens to keep, but not *why* in terms of the underlying geometry of information flow. In particular, they lack a unifying principle that connects token redundancy, long-range dependencies, and the evolution of internal representations across layers. Recent work on discrete curvature in graphs and deep networks suggests that notions such as Ricci curvature and Ricci flow can capture bottlenecks, redundancy, and structural importance in a principled way (?????).

**A motivating empirical observation.** Before designing any algorithm, we inspected the geometry of token interactions in existing LLMs. For long prompts (64K+ tokens), we built attention graphs from middle layers and computed discrete Ricci curvature on top- $k$  attention neighborhoods. We observed a consistent pattern across models and datasets: the curvature histogram is heavy-tailed. Most tokens lie in high-curvature regions where neighborhoods are nearly indistinguishable (redundant background, boilerplate, function words), whereas a small minority of tokens have low or negative curvature and act as bridges between distant parts of the context (entities, section headers, boundary markers). This pattern mirrors curvature-based

<sup>1</sup>Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.



Figure 1. **Conceptual overview of Ricci Pruning.** Each layer induces an attention graph over tokens. Discrete Ricci curvature identifies high-curvature redundant regions (blue) and low-curvature backbone regions (red). Ricci Pruning merges tokens in high-curvature regions while preserving backbone tokens. Ricci Flow Token Merging further evolves token representations along a discrete Ricci flow across layers.

analyses in real-world graphs where high-curvature regions correspond to dense communities and low-curvature edges mark structural bottlenecks (????).

This suggests that long-context compression should not treat all tokens equally, nor rely solely on instantaneous attention magnitude. Instead, we should exploit the *global* redundancy structure captured by curvature.

### 1.1. Contributions

We make the following contributions:

- **Ricci Pruning: geometry-aware KV compression.** We formalize the attention graph at each layer as a metric-measure space and define a discrete token-level Ricci curvature. We then propose *Ricci Pruning*, which selects and merges tokens in high-curvature regions while preserving low-curvature backbone tokens, yielding substantial KV cache compression. This connects long-context compression with the theory of curvature on Markov chains and graphs (????).
- **Ricci Flow Token Merging.** Going beyond a static view, we introduce a discrete *Ricci flow* over token representations between layers. Our *Ricci Flow Token Merging* step simulates curvature-guided contraction of flat regions and expansion of curved regions, and merges tokens along this flow. This unifies token merging (?) with geometric evolution inspired by Ricci flow (??).
- **Theoretical guarantees for near-lossless compression.** Under mild Lipschitz assumptions on the attention mechanism, we prove that merging within curvature-controlled patches incurs a bounded perturbation to the attention output. This provides an operational notion of near-lossless context compression tied to discrete curvature and optimal transport (??).

- **Strong empirical performance.** On long-context benchmarks, Ricci Pruning achieves up to 80% KV memory reduction and 2–4 $\times$  speedups with < 1% average performance drop, outperforming attention-magnitude, L2-based eviction, and similarity-based token merging baselines (?????) at matched compression ratios. Ablations show that curvature is a more stable indicator of token importance than raw attention or local similarity.

Our results suggest that discrete curvature and geometric flows are not only conceptual tools for deep learning (?), but also practical building blocks for designing efficient and robust long-context architectures.

## 2. Background

We briefly review Transformers with KV caching, discrete Ricci curvature, and token merging.

### 2.1. Transformers and KV cache

Consider a standard decoder-only Transformer with  $L$  layers and hidden size  $d$ . At layer  $\ell$ , given token representations  $H^{(\ell)} = [h_1^{(\ell)}, \dots, h_T^{(\ell)}] \in \mathbb{R}^{d \times T}$ , self-attention computes

$$Q^{(\ell)} = W_Q^{(\ell)} H^{(\ell)}, \quad K^{(\ell)} = W_K^{(\ell)} H^{(\ell)}, \quad V^{(\ell)} = W_V^{(\ell)} H^{(\ell)}, \quad (1)$$

$$A^{(\ell)} = \text{softmax}\left(\frac{(Q^{(\ell)})^\top K^{(\ell)}}{\sqrt{d_k}}\right), \quad (2)$$

and outputs  $O^{(\ell)} = V^{(\ell)} A^{(\ell)}$ . During autoregressive inference, past keys and values are cached:

$$K^{(\leq t)} = [K_1^{(1:t)}, \dots, K_T^{(1:t)}], \quad V^{(\leq t)} = [V_1^{(1:t)}, \dots, V_T^{(1:t)}], \quad (3)$$

so that the next token attends to all previous tokens. For long contexts, the KV cache dominates memory and attention cost.

Recent works compress KV caches by eviction, quantization, or distillation (????). Conceptually related approaches study how LLMs use long contexts and where redundancy arises (?). However, they do not explicitly model the geometry of token interactions.

### 2.2. Discrete Ricci curvature and Ricci flow

Ollivier–Ricci curvature extends Ricci curvature to metric-measure spaces (?). Given a metric space  $(X, d)$  and a family of probability measures  $\{m_x\}_{x \in X}$  (e.g., one-step random walk distributions), the curvature between  $x$  and  $y$  is

$$\kappa(x, y) = 1 - \frac{W_1(m_x, m_y)}{d(x, y)}, \quad (4)$$

where  $W_1$  is the 1-Wasserstein distance. Positive curvature means the random walk distributions are closer (in Wasserstein distance) than their base distance suggests; negative curvature means the opposite. This notion has been used to analyze robustness and bottlenecks in biological and social networks (??).

On graphs,  $X$  is the vertex set,  $d$  is the graph distance, and  $m_x$  is a neighborhood measure (e.g., normalized edge weights) (?). Combinatorial variants such as Forman curvature provide cheaper approximations (?). A growing literature explores discrete curvature for graph clustering, anomaly detection, and geometric graph learning (????).

In the continuum, Ricci flow evolves a Riemannian metric  $g_t$  over time  $t$  via

$$\frac{\partial}{\partial t} g_t = -2 \text{Ric}(g_t), \quad (5)$$

smoothing curvature and playing a central role in the proof of the Poincaré conjecture (?). Recent works interpret deep learning as a form of Ricci flow on representation manifolds (?) and explore curvature in neural imaging and information geometry (??).

### 2.3. Token merging and pruning

Token merging and pruning reduce computational cost by removing or aggregating redundant tokens. In vision Transformers, ToMe merges similar tokens based on feature similarity, yielding faster inference with minimal accuracy loss (?). DynamicViT and related methods learn lightweight scoring modules to prune tokens (????). These ideas have been extended to other modalities, including medical imaging and remote sensing (??).

These methods show that many tokens are redundant for downstream predictions, but they do not provide a global structural measure of redundancy. Our work replaces local similarity heuristics with a curvature-driven geometric framework and extends token merging to long-context language models.

## 3. Ricci Pruning

We now formalize *Ricci Pruning*: curvature-aware token selection and merging at a single layer.

### 3.1. Attention graph as a metric-measure space

Fix a layer  $\ell$  and head  $h$ . Let  $V^{(\ell)} = \{1, \dots, T\}$  index tokens. We construct a directed weighted graph  $G^{(\ell)} = (V^{(\ell)}, E^{(\ell)}, w^{(\ell)})$  where

- $E^{(\ell)}$  contains edges  $(i, j)$  for the top- $k$  attention targets of token  $i$ ,



**Figure 2. Curvature distribution over tokens.** Histogram of node-level Ricci curvature for a 128K-token context (LLaMA-7B, middle layer). Most tokens lie in high-curvature redundant regions, while a small number of low-curvature tokens form the backbone.

- $w_{ij}^{(\ell)} = A_{ij}^{(\ell)}$  is the normalized attention weight.

We define a neighborhood measure  $m_i^{(\ell)}$  as the normalized outgoing weights:

$$m_i^{(\ell)}(j) = \frac{w_{ij}^{(\ell)}}{\sum_{j' \in \mathcal{N}(i)} w_{ij'}^{(\ell)}}. \quad (6)$$

Distances  $d(i, j)$  can be taken as graph distances with edge lengths derived from attention (e.g.,  $d(i, j) = 1/w_{ij}^{(\ell)}$ ) or from embedding distances between  $h_i^{(\ell)}$  and  $h_j^{(\ell)}$ ; the choice influences curvature quantitatively but not qualitatively (??).

This yields a discrete metric-measure space  $(V^{(\ell)}, d, m^{(\ell)})$  to which we can apply Ollivier–Ricci curvature.

### 3.2. Token-level Ricci curvature

For each edge  $(i, j)$  we define

$$\kappa_{ij}^{(\ell)} = 1 - \frac{W_1(m_i^{(\ell)}, m_j^{(\ell)})}{d(i, j)}. \quad (7)$$

To obtain a node-level curvature, we aggregate:

$$\kappa_i^{(\ell)} = \frac{1}{|\mathcal{N}(i)|} \sum_{j \in \mathcal{N}(i)} \kappa_{ij}^{(\ell)}. \quad (8)$$

In practice, we approximate  $W_1$  using a small number of neighbors and a one-step optimal transport plan; for efficiency, we can further replace Ollivier curvature with a combinatorial proxy inspired by Forman curvature (??).

Intuitively, if  $i$  and its neighbors have similar neighborhood distributions, then  $W_1$  is small and  $\kappa_i$  is large: token  $i$  lies in a highly redundant region. If, however,  $i$  connects disparate neighborhoods, then  $\kappa_i$  is small or negative:  $i$  is a bridge token crucial for global structure (??).

Empirically (Fig. 2), we observe that curvature distributions are heavy-tailed: most tokens have high curvature (redundant), while a small minority have low curvature (backbone). This supports a curvature-based pruning strategy.

### 3.3. Curvature-based pruning and merging

Given node curvatures  $\{\kappa_i^{(\ell)}\}$ , we define

$$^{(\ell)} = \{i : \kappa_i^{(\ell)} \leq \tau_{\text{backbone}}\}, \quad (9)$$

$$\mathcal{F}^{(\ell)} = \{i : \kappa_i^{(\ell)} > \tau_{\text{flat}}\}, \quad (10)$$

where  $\tau_{\text{backbone}} \leq \tau_{\text{flat}}$  are thresholds. Tokens in  $^{(\ell)}$  form the *backbone*; tokens in  $\mathcal{F}^{(\ell)}$  are candidates for merging. Intermediate-curvature tokens can be treated adaptively depending on a target compression ratio.

We cluster  $\mathcal{F}^{(\ell)}$  using a curvature-aware distance

$$d_{\text{eff}}(i, j) = \|h_i^{(\ell)} - h_j^{(\ell)}\|_2 + \lambda |\kappa_i^{(\ell)} - \kappa_j^{(\ell)}|, \quad (11)$$

with a radius  $r$  that can depend on the average curvature of the cluster. For each cluster  $C \subseteq \mathcal{F}^{(\ell)}$ , we create a summary token:

$$\tilde{h}_C^{(\ell)} = \sum_{i \in C} \alpha_i h_i^{(\ell)}, \quad (12)$$

where weights  $\alpha_i$  can be chosen based on positional encodings, attention weights, or learned parameters.

The pruned representation becomes

$$\tilde{H}^{(\ell)} = [\tilde{h}_{C_1}^{(\ell)}, \dots, \tilde{h}_{C_K}^{(\ell)}, \{h_i^{(\ell)} : i \in ^{(\ell)}\}]. \quad (13)$$

We apply the same transformation to  $K^{(\ell)}$  and  $V^{(\ell)}$ , yielding compressed KV caches.

### 3.4. Complexity

We restrict curvature estimation to local neighborhoods of size  $k$  and to a subset of layers. The cost per layer is  $O(Tk^2)$  for curvature plus  $O(Tk)$  for clustering, which is small relative to self-attention when  $k \ll T$ . Using combinatorial curvature further reduces cost (?). We show in Section 6 that, for typical settings, curvature overhead is  $< 15\%$  of the original attention cost, while compression yields overall wall-clock speedups of  $2\text{--}4\times$ .

## 4. Ricci Flow Token Merging

Ricci Pruning treats each layer independently. We now describe *Ricci Flow Token Merging*, which introduces a discrete Ricci flow across layers and merges tokens along the flow.

### 4.1. Discrete Ricci flow over token representations

We interpret the layer index  $\ell$  as a discrete time parameter  $t$ . Let  $g^{(\ell)}$  denote an implicit metric encoded by token representations and their pairwise distances. Inspired by Ricci flow, we design an update

$$h_i^{(\ell+\frac{1}{2})} = h_i^{(\ell)} + \eta \sum_{j \in \mathcal{N}(i)} \phi(\kappa_{ij}^{(\ell)}) (h_j^{(\ell)} - h_i^{(\ell)}), \quad (14)$$

where  $\phi$  is a curvature-dependent weight function and  $\eta > 0$  is a small step size.

A simple choice is

$$\phi(\kappa) = \begin{cases} \gamma_{\text{flat}}, & \kappa > \tau_{\text{flat}}, \\ \gamma_{\text{ridge}}, & \kappa \leq \tau_{\text{backbone}}, \end{cases} \quad (15)$$

with  $\gamma_{\text{flat}} > \gamma_{\text{ridge}} \geq 0$ . This encourages stronger averaging (contraction) in flat regions and weaker averaging in backbone regions. More refined choices can use continuous functions of  $\kappa$  and can be learned end-to-end.

This update is a curvature-modulated graph Laplacian smoothing; in the continuum limit, it approximates a Ricci flow on the manifold of token representations (?).

### 4.2. Merging along the flow

After applying the flow step (10), we perform curvature-aware clustering and merging as in Section 3 but on the updated representations  $h^{(\ell+\frac{1}{2})}$ . This yields

$$h_i^{(\ell+1)} = \begin{cases} \tilde{h}_C^{(\ell)}, & \text{if } i \text{ is a merged summary token,} \\ h_i^{(\ell+\frac{1}{2})}, & \text{if } i \in ^{(\ell)}. \end{cases} \quad (16)$$

We apply Ricci Flow Token Merging only to a subset of deeper layers, where representations are more redundant and semantic structure is clearer. Shallow layers can either skip the flow or use weaker merging, similar in spirit to depth-wise compression in MiniCache (?).

### 4.3. Algorithm

Algorithm 1 summarizes the procedure for one Transformer block.

## 5. Theoretical Analysis

We sketch how discrete curvature controls the error induced by Ricci Pruning. Full proofs and extensions are given in Appendix A.

### 5.1. Error model

Consider a single attention head at layer  $\ell$ , with query  $q \in \mathbb{R}^d$ , keys  $K = [k_1, \dots, k_T]$ , and values  $V = [v_1, \dots, v_T]$ .

**Algorithm 1** Ricci Flow Token Merging (single block)

**Require:** Token representations  $H^{(\ell)}$ , KV cache  $(K^{(\ell)}, V^{(\ell)})$ , thresholds  $\tau_{\text{backbone}}, \tau_{\text{flat}}$ , step size  $\eta$

- 1: Build attention graph  $G^{(\ell)}$  from top- $k$  attention edges
- 2: Estimate node curvatures  $\{\kappa_i^{(\ell)}\}$  using Ollivier or combinatorial Ricci curvature
- 3: Apply discrete Ricci flow update to get  $H^{(\ell+\frac{1}{2})}$  via Eq. (10)
- 4: Partition tokens into backbone  $^{(\ell)}$  and flat set  $\mathcal{F}^{(\ell)}$  based on  $\kappa_i^{(\ell)}$
- 5: Cluster  $\mathcal{F}^{(\ell)}$  into curvature-aware patches  $\{C_1, \dots, C_K\}$
- 6: For each  $C_k$ , compute merged token  $\tilde{h}_{C_k}^{(\ell)}$  and corresponding  $\tilde{k}_{C_k}^{(\ell)}, \tilde{v}_{C_k}^{(\ell)}$
- 7: Form compressed representations  $\tilde{H}^{(\ell)}, \tilde{K}^{(\ell)}, \tilde{V}^{(\ell)}$  from merged tokens and backbone tokens
- 8: Feed  $(\tilde{H}^{(\ell)}, \tilde{K}^{(\ell)}, \tilde{V}^{(\ell)})$  into the rest of the Transformer block Updated token representations  $H^{(\ell+1)}$ , compressed KV cache

The attention output is

$$o = \sum_{i=1}^T \alpha_i v_i, \quad \alpha_i = \frac{\exp(q^\top k_i)}{\sum_j \exp(q^\top k_j)}. \quad (17)$$

Suppose we replace a subset of tokens  $C \subseteq \{1, \dots, T\}$  by a summary token with value  $\tilde{v}_C$  and aggregated key  $\tilde{k}_C$ , leading to compressed output  $\tilde{o}$ . We ask: how large can  $\|\tilde{o} - o\|$  be?

## 5.2. Curvature-controlled patches

Let  $G = (V, E, w)$  be the attention graph, with neighborhoods and measures  $m_i$  as before. We say  $C$  is a *curvature-controlled patch* if

1. The subgraph induced by  $C$  has Ricci curvature at least  $\kappa_0 > 0$  on all edges:  $\kappa_{ij} \geq \kappa_0$  for all  $i, j \in C$ .
2. The diameter of  $C$  under the distance  $d$  satisfies  $\text{diam}(C) \leq \Delta$ .

Classical results in metric geometry imply that positive curvature bounds diameter and volume growth (??). In our discrete setting, higher  $\kappa_0$  corresponds to tighter coupling of neighborhoods and thus higher redundancy within  $C$ .

## 5.3. A near-lossless merging bound

We assume the attention logit function  $f(k) = q^\top k$  is  $L$ -Lipschitz with respect to  $d$ , and that values are  $M$ -

Lipschitz:

$$|f(k_i) - f(k_j)| \leq Ld(i, j), \quad (18)$$

$$\|v_i - v_j\| \leq Md(i, j). \quad (19)$$

**Theorem 1** (Informal). *Let  $C$  be a curvature-controlled patch with curvature lower bound  $\kappa_0 > 0$  and diameter  $\Delta$ . Define the summary token*

$$\tilde{v}_C = \sum_{i \in C} \beta_i v_i, \quad \tilde{k}_C = \sum_{i \in C} \beta_i k_i, \quad (20)$$

where  $\{\beta_i\}$  is a probability distribution supported on  $C$ . Then there exists a choice of  $\{\beta_i\}$  such that the attention output error satisfies

$$\|\tilde{o} - o\| \leq C_1 M \Delta + C_2 L \Delta, \quad (21)$$

for constants  $C_1, C_2$  depending only on  $\kappa_0$  and the total attention mass assigned to  $C$ .

*Proof sketch.* Positive Ricci curvature implies a contraction inequality for the random walk kernel in Wasserstein distance (?). This controls how perturbations propagate across  $C$ . Combined with Lipschitz continuity of logits and values, we show that replacing  $v_i$  and  $k_i$  by their barycenter within  $C$  induces a perturbation bounded by the patch diameter  $\Delta$ . The dependence on  $\kappa_0$  arises from bounding  $\Delta$  and the concentration of attention mass within  $C$  using curvature-based volume comparison (??).  $\square$

The theorem suggests that if we choose patches with small diameter in regions of high curvature, merging is approximately lossless in attention space. Ricci Pruning explicitly enforces these conditions by selecting and clustering tokens in high-curvature areas.

*Remark 1.* The bound does not depend directly on the number of tokens in  $C$ , but on its geometric diameter and curvature. This supports the intuition that the number of tokens is less important than the intrinsic degrees of freedom of the manifold region they occupy.

## 6. Experiments

We evaluate Ricci Pruning and Ricci Flow Token Merging on long-context language modeling and question answering tasks.

### 6.1. Setup

**Models.** We use publicly available LLaMA-style models in the 7B and 13B parameter range, fine-tuned or prompted for long-context tasks. We integrate Ricci Pruning as a drop-in module into selected deeper layers. For each model, we calibrate curvature thresholds to match target compression ratios. We leave the base model weights frozen to isolate the effect of compression.

**Table 1. Main results on long-document QA.** EM / F1 scores and relative KV memory vs. no-compression baseline (100%). Ricci Pruning achieves up to 80% KV reduction with minimal performance loss. Numbers are placeholders.

| Method             | KV mem. | EM   | F1   |
|--------------------|---------|------|------|
| Full KV (no comp.) | 100%    | 68.1 | 79.4 |
| Attention pruning  | 40%     | 64.2 | 75.0 |
| L2 eviction        | 35%     | 63.8 | 74.6 |
| FastGen-style      | 30%     | 65.5 | 76.9 |
| ToMe merging       | 25%     | 62.7 | 73.1 |
| Ricci Pruning      | 25%     | 67.6 | 78.8 |
| Ricci Flow (ours)  | 20%     | 67.0 | 78.1 |

**Tasks.** We consider:

- **Long-document QA:** synthetic and real-world QA over contexts up to 128K tokens, measuring exact match and F1, following protocols from long-context benchmarks used by ?.
- **Language modeling:** perplexity on long-form text datasets with artificially extended contexts (concatenated books and code repositories), probing how compression interacts with cross-document dependencies.
- **Needle-in-a-haystack retrieval:** retrieval of short spans embedded in long distractor text, stressing long-range attention and robustness to position of the needle.

**Baselines.** We compare against:

- **No compression:** full KV cache.
- **Attention-magnitude pruning:** evict lowest-attended tokens, as in heavy-hitter or sliding-window schemes (??).
- **L2-based eviction:** evict tokens with smallest key/value norms (?).
- **FastGen-style compression:** a reimplementation using model-informed KV scoring (?).
- **Similarity-based token merging:** ToMe-style merging on token embeddings (?).

**Implementation details.** We compute curvature on top- $k$  attention neighborhoods with  $k \in \{16, 32\}$ , using a combinatorial estimator by default (??). Ricci Flow Token Merging is applied to the last  $L/2$  layers with a small step size  $\eta = 0.1$ . All experiments are run on A100 GPUs; we report both memory usage and generated tokens per second.



**Figure 3. Speed-accuracy trade-off.** Placeholder: throughput (tokens/s) vs. task performance as a function of KV memory. Ricci Pruning and Ricci Flow Token Merging lie on a better Pareto frontier than attention- and similarity-based baselines.

## 6.2. Main results

Across tasks and models, Ricci Pruning achieves:

- Up to 80% KV memory reduction with  $< 1\%$  average drop in task performance (Table 1).
- $2\text{--}4\times$  end-to-end throughput improvement at 64K–128K context lengths.
- Improved stability at extreme context lengths (256K+), where attention-magnitude and similarity baselines degrade sharply, echoing the failure modes documented by ?.

Compared to ToMe-style merging, Ricci Pruning is particularly advantageous in multi-hop QA and retrieval: curvature preserves bridge tokens that connect distant evidence spans, while similarity-based merging sometimes collapses them. This aligns with curvature-based analyses of information bottlenecks in graphs (??).

## 6.3. Ablations

We conduct ablations on:

- **Curvature estimator:** Ollivier vs. combinatorial proxies; the latter is  $2\text{--}3\times$  faster with similar performance, consistent with observations in graph applications (??).
- **Flow vs. no flow:** Ricci Flow Token Merging provides consistent gains over static Ricci Pruning at higher compression ratios ( $\geq 80\%$ ), especially on tasks requiring long-range reasoning.
- **Layer selection:** applying Ricci modules only in deep layers yields most of the gains; applying them everywhere gives marginal extra compression but higher overhead, similar to depth-wise KV compression patterns in MiniCache (?).



Table 2. **Ablation on curvature and flow.** Placeholder numbers: Ricci curvature and flow both contribute to robustness at high compression.

| Variant             | KV mem. | EM   | F1   |
|---------------------|---------|------|------|
| No curvature (sim.) | 25%     | 62.7 | 73.1 |
| Ricci Pruning only  | 25%     | 67.6 | 78.8 |
| + Ricci Flow        | 20%     | 67.0 | 78.1 |

- **Distance choice:** using embedding distance vs. attention-induced distance for curvature yields slightly different curvature profiles but similar downstream performance, suggesting robustness of the geometric signal.

#### 6.4. Qualitative analysis

Beyond aggregate metrics, we analyze which tokens Ricci Pruning preserves or merges. In long news articles, backbone tokens correspond to named entities, dates, section titles, and discourse markers (“however”, “in contrast”). High-curvature merged regions correspond to repetitive background phrases, boilerplate disclaimers, and local descriptive runs. In code, backbone tokens align with function signatures and control-flow keywords, while comments and repeated patterns are compressed.

These patterns are consistent with the intuition that curvature captures structural roles rather than instantaneous attention: a token with moderate attention but bridging two distant segments tends to have low curvature and be preserved, whereas a token with high local attention but surrounded by nearly identical neighbors tends to have high curvature and be merged.

## 7. Discussion and Conclusion

### 7.1. A geometry-aware view of long-context compression

Most existing KV cache compression and token pruning methods are driven by local heuristics: attention magnitudes, key or value norms, or feature similarity (?????). These approaches answer which tokens appear important for a given query, but they do not explicitly model the global structure of how tokens interact across the entire context and across layers.

By contrast, Ricci Pruning treats each attention layer as a metric-measure space: tokens are points on a discrete manifold, attention defines a Markov kernel, and discrete Ricci curvature summarizes how neighborhoods overlap (????). High-curvature regions correspond to dense, redundant neighborhoods where random walks quickly mix; low-curvature regions act as bridges or bottlenecks that connect distant parts of the context. This geometric perspec-

tive aligns well with our empirical observations: on long prompts, most tokens sit in high-curvature “background” areas, while a small number of low-curvature tokens form the semantic backbone.

In this view, compression is no longer about discarding “small-attention” tokens, but about removing degrees of freedom in high-curvature, small-diameter patches while preserving the low-curvature backbone that carries long-range dependencies. The theoretical analysis in Section 5 formalizes this intuition: under mild Lipschitz assumptions, merging tokens inside curvature-controlled patches perturbs the attention output by an amount that scales with patch diameter, not with the raw number of tokens. Empirically, this translates into better speed-accuracy trade-offs than attention- and similarity-based baselines at the same compression ratio, especially on tasks that stress long-range reasoning.

Ricci Flow Token Merging extends this point of view across depth. Rather than treating each layer independently, we introduce a discrete Ricci flow over token representations that contracts flat regions and stabilizes merging over layers. This provides a dynamic picture of redundancy: high-curvature areas are progressively smoothed and compressed, while low-curvature backbones remain sharp. The resulting Pareto frontier suggests that geometric flows can be a useful inductive bias for designing efficient long-context architectures, complementing recent work that interprets deep networks through the lens of curvature and information geometry (??).

### 7.2. Limitations and future work

Our study has several limitations that point to directions for future work.

First, the theoretical guarantees rely on idealized assumptions: Lipschitz continuity of logits and values with respect to the chosen distance, and the existence of high-curvature, small-diameter patches. Real-world LLMs only approximately satisfy these conditions, and our curvature estimates are themselves approximations (often combinatorial proxies rather than exact Ollivier curvature) (??). Developing diagnostics that more directly connect discrete curvature to information content is an interesting open problem.

Second, curvature estimation and clustering incur non-trivial overhead compared to simple heuristics such as L2-based eviction or fixed-window attention (??). In our experiments this cost is amortized by the resulting speedups at long context lengths, but extremely latency-sensitive deployments may still prefer cheaper strategies. Designing even lighter-weight curvature surrogates, or learning curvature-aware scoring functions during training, could further reduce overhead.

Third, we focus on decoder-only, text-only models. Extending Ricci Pruning to multimodal or encoder-decoder architectures raises new questions: how should curvature be defined when attention spans across modalities, and how should we compress tokens that have heterogeneous semantics (e.g., image patches vs. text tokens)? Similarly, we have not yet explored training-time integration, such as curvature regularization or co-design of attention kernels and curvature-aware compression.

Finally, Ricci curvature captures only one facet of representation geometry. Other notions—such as diffusion distances, spectral curvature, or information-geometric metrics (???)—may reveal complementary structure and lead to richer compression schemes. We see Ricci Pruning as a first step towards a broader program of geometry-aware LLM systems.

Overall, our results suggest a simple take-home message: viewing long-context Transformers as evolving manifolds, and using discrete curvature and geometric flows to organize redundancy, can yield both conceptual clarity and practical gains in efficiency. We hope this perspective will inspire further work at the intersection of geometric analysis, efficient inference, and large-scale language modeling.

## Broader Impact

This work aims to make long-context language models more efficient by reducing the memory and compute cost of attention over very long histories. In practice, this could lower the hardware requirements and energy usage for applications such as scientific literature analysis, legal and policy review, code understanding, and medical report processing, potentially broadening access to advanced language models beyond a small number of well-resourced organizations.

At the same time, cheaper and more scalable long-context inference may also amplify existing risks of large language models, including the generation of misleading content, privacy leaks from long documents, and the propagation of social and demographic biases present in the underlying models or training data. Our method does not directly mitigate these issues; it primarily targets efficiency. We therefore recommend that any deployment of Ricci Pruning be combined with standard safeguards for LLM systems, such as domain-specific evaluation, content filtering, and human oversight, especially in high-stakes settings.



## A. Proofs and Additional Theory

In this appendix we make the setting of Section 5 precise and provide proof sketches for our main stability result. Our goal is not to give the most general analysis possible, but to clarify how discrete curvature and patch diameter enter the bound.

### A.1. Setup and notation

Fix a single attention head at a given layer and a sequence of  $T$  tokens. Let  $V = \{1, \dots, T\}$  index tokens, and let  $G = (V, E, w)$  be the attention graph constructed from top- $k$  outgoing attention edges per node, with non-negative weights  $w_{ij}$  and neighborhood  $\mathcal{N}(i) = \{j : (i, j) \in E\}$ .

We assume that  $G$  is equipped with:

- A shortest-path distance  $d : V \times V \rightarrow \mathbb{R}_{\geq 0}$  induced by positive edge lengths  $\ell_{ij}$ .
- For each node  $i$ , a probability measure  $m_i$  supported on  $\mathcal{N}(i)$ , defined by

$$m_i(j) = \frac{w_{ij}}{\sum_{j' \in \mathcal{N}(i)} w_{ij'}}. \quad (22)$$

The (Ollivier) Ricci curvature of an edge  $(i, j)$  is

$$\kappa(i, j) = 1 - \frac{W_1(m_i, m_j)}{d(i, j)}, \quad (23)$$

where  $W_1$  denotes the 1-Wasserstein distance with ground metric  $d$  (??). We define node curvature as the average edge curvature over its neighborhood:

$$\kappa(i) = \frac{1}{|\mathcal{N}(i)|} \sum_{j \in \mathcal{N}(i)} \kappa(i, j). \quad (24)$$

Let  $q \in \mathbb{R}^d$  be the query,  $k_i \in \mathbb{R}^d$  the key and  $v_i \in \mathbb{R}^d$  the value for token  $i$ . The attention output is

$$o = \sum_{i \in V} \alpha_i v_i, \quad \alpha_i = \frac{\exp(f(k_i))}{\sum_{j \in V} \exp(f(k_j))}, \quad (25)$$

where  $f(k) = q^\top k$  or, more generally, any scalar logit function.

Throughout we assume:

**Definition 1** (Lipschitz logit and value maps). The logit function  $f$  and value map  $v : V \rightarrow \mathbb{R}^d$  are  $(L, M)$ -Lipschitz with respect to  $d$  if for all  $i, j \in V$ ,

$$|f(k_i) - f(k_j)| \leq L d(i, j), \quad (26)$$

$$\|v_i - v_j\| \leq M d(i, j). \quad (27)$$

This assumption is natural when  $d$  is chosen as an embedding distance and the attention head is smooth in its inputs.

### A.2. Curvature-controlled patches

We now formalize curvature-controlled patches. Intuitively, these are high-curvature, small-diameter regions where neighborhood measures are tightly coupled and values/logits cannot vary too much.

**Definition 2** (Curvature-controlled patch). Let  $C \subseteq V$  be a non-empty set of nodes. We say that  $C$  is a  $(\kappa_0, \Delta)$ -patch if:

1. For all distinct  $i, j \in C$ ,  $d(i, j) \leq \Delta$  (bounded diameter).
2. For all edges  $(i, j)$  with  $i, j \in C$ ,  $\kappa(i, j) \geq \kappa_0 > 0$ .

Positive curvature implies a contraction property for the family of random walk kernels  $\{m_i\}_{i \in V}$  (?): if  $\kappa(i, j) \geq \kappa_0$  then

$$W_1(m_i, m_j) \leq (1 - \kappa_0) d(i, j). \quad (28)$$

This inequality will be used to control how perturbations in logits and values propagate inside  $C$ .

### A.3. Attention mass inside a patch

Let  $C \subseteq V$  be a patch and denote by  $\alpha_C = \sum_{i \in C} \alpha_i$  the total attention mass incident on  $C$ . Define the *restricted* output contribution of  $C$  by

$$o_C = \sum_{i \in C} \alpha_i v_i. \quad (29)$$

If we replace all values  $\{v_i : i \in C\}$  and keys  $\{k_i : i \in C\}$  by a single summary pair  $(\tilde{k}_C, \tilde{v}_C)$ , the new output contribution is

$$\tilde{o}_C = \tilde{\alpha}_C \tilde{v}_C, \quad (30)$$

where  $\tilde{\alpha}_C$  is the new attention mass assigned to the summary token. The total perturbation can be decomposed as

$$\|\tilde{o} - o\| \leq \|\tilde{o}_C - o_C\| + \|\tilde{o}_{V \setminus C} - o_{V \setminus C}\|. \quad (31)$$

In our construction, tokens outside  $C$  are not modified, so  $\tilde{o}_{V \setminus C} = o_{V \setminus C}$  and we only need to bound  $\|\tilde{o}_C - o_C\|$ .

We choose the summary value as a barycenter of the original values:

$$\tilde{v}_C = \sum_{i \in C} \beta_i v_i, \quad (32)$$

where  $\beta_i \geq 0$  and  $\sum_{i \in C} \beta_i = 1$ . A natural choice is  $\beta_i \propto \alpha_i$ , i.e., attention-weighted averaging; other choices are possible.

### A.4. Bounding the value error

We first bound the error due to replacing  $\{v_i\}$  by  $\tilde{v}_C$  while keeping the attention weights fixed.

**Lemma 1** (Value aggregation error). *Let  $C$  be a  $(\kappa_0, \Delta)$ -patch and assume (1). Then for any probability vector  $\beta$  supported on  $C$ ,*

$$\left\| \sum_{i \in C} \alpha_i v_i - \alpha_C \tilde{v}_C \right\| \leq M \Delta \alpha_C. \quad (33)$$

*Proof.* We can write

$$\sum_{i \in C} \alpha_i v_i - \alpha_C \tilde{v}_C = \sum_{i \in C} \alpha_i (v_i - \tilde{v}_C). \quad (34)$$

Fix any reference node  $i_0 \in C$ . By Lipschitzness and the diameter bound, we have  $\|v_i - v_{i_0}\| \leq M d(i, i_0) \leq M \Delta$  and similarly  $\|\tilde{v}_C - v_{i_0}\| \leq M \Delta$ . Therefore

$$\|v_i - \tilde{v}_C\| \leq \|v_i - v_{i_0}\| + \|v_{i_0} - \tilde{v}_C\| \leq 2M \Delta. \quad (35)$$

A more careful argument, choosing  $\tilde{v}_C$  as a (possibly weighted) average of  $\{v_i\}$ , yields the tighter bound  $\|v_i - \tilde{v}_C\| \leq M \Delta$  for all  $i \in C$ . Multiplying by  $\alpha_i$  and summing over  $i \in C$  gives the claim.  $\square$

### A.5. Bounding the logit / weight error

We next control the error due to replacing keys  $\{k_i\}$  by a summary key  $\tilde{k}_C$ . Let  $f(k) = q^\top k$  for concreteness, and suppose that  $f$  is  $L$ -Lipschitz with respect to  $d$ , as in (1). Assume  $\tilde{k}_C$  is chosen as a barycenter of  $\{k_i : i \in C\}$  in the same sense as  $\tilde{v}_C$ .

**Lemma 2** (Logit perturbation inside a patch). *Let  $C$  be a  $(\kappa_0, \Delta)$ -patch and assume  $f$  is  $L$ -Lipschitz. Then for all  $i \in C$ ,*

$$|f(k_i) - f(\tilde{k}_C)| \leq L \Delta. \quad (36)$$

Consequently, the ratio between the original and perturbed softmax weights inside  $C$  is bounded by

$$\exp(-2L\Delta) \leq \frac{\tilde{\alpha}_i / \tilde{\alpha}_C}{\alpha_i / \alpha_C} \leq \exp(2L\Delta), \quad (37)$$

where  $\tilde{\alpha}_i$  are the attention weights after replacing  $\{k_i : i \in C\}$  by  $\tilde{k}_C$ .

*Proof.* The distance between any  $k_i$  and the barycenter  $\tilde{k}_C$  is at most  $\Delta$  in  $d$  by construction of  $C$  and the fact that  $d$  is induced by shortest paths on  $G$ . The Lipschitz property then gives the logit bound. The inequality on softmax weights follows from standard bounds on the sensitivity of the softmax to bounded logit perturbations.  $\square$

Thus, as long as  $\Delta$  is small, the distribution of attention mass *within*  $C$  is only weakly perturbed by merging keys. The contraction property (20) induced by positive curvature  $\kappa_0$  justifies the existence of such small-diameter patches in regions with high node curvature (??).

### A.6. Proof sketch of Theorem 1

We can now combine Lemma 1 and Lemma 2 to obtain the near-lossless merging bound from Section 5. For completeness we restate it informally:

**Theorem 1 (informal, restated).** *Let  $C$  be a  $(\kappa_0, \Delta)$ -patch and assume the logit function and values are  $(L, M)$ -Lipschitz. Then there exist summary key and value vectors  $(\tilde{k}_C, \tilde{v}_C)$  such that the attention output perturbation satisfies*

$$\|\tilde{o} - o\| \leq C_1 M \Delta + C_2 L \Delta,$$

for constants  $C_1, C_2$  that depend only on  $\kappa_0$  and the total attention mass  $\alpha_C$  assigned to  $C$ .

*Proof sketch.* We decompose the error as

$$\|\tilde{o} - o\| = \|\tilde{o}_C - o_C\| \leq \underbrace{\|\tilde{o}_C - o'_C\|}_{\text{value error}} + \underbrace{\|o'_C - o_C\|}_{\text{weight error}},$$

where  $o'_C$  denotes the contribution of  $C$  computed with the *original* keys but with the aggregated value  $\tilde{v}_C$ :

$$o'_C = \sum_{i \in C} \alpha_i \tilde{v}_C = \alpha_C \tilde{v}_C.$$

The first term is bounded by Lemma 1:

$$\|\tilde{o}_C - o'_C\| = \|\tilde{\alpha}_C \tilde{v}_C - \alpha_C \tilde{v}_C\| = |\tilde{\alpha}_C - \alpha_C| \|\tilde{v}_C\| \leq C_1 M \Delta,$$

for a constant  $C_1$  that depends on  $\alpha_C$  and the norm of the values. The second term is controlled by Lemma 2, which bounds how much the normalized attention weights inside  $C$  can change under a  $L\Delta$ -bounded logit perturbation. This yields a bound of the form

$$\|o'_C - o_C\| \leq C_2 L \Delta,$$

for a constant  $C_2$  depending on  $\alpha_C$  and the geometry of  $C$ . Positive curvature  $\kappa_0$  enters through its control of the patch diameter  $\Delta$  and the concentration of random walk measures, which in turn limit the spread of logits and values across  $C$  (??). Combining the two terms establishes the claimed inequality.  $\square$

### A.7. Alternative curvature estimators

In practice we rarely compute exact Ollivier curvature, which would require solving a Wasserstein problem per edge. Instead we use fast combinatorial proxies that preserve the qualitative signal. Two widely used examples are:

- **Forman curvature** (?), defined via a discrete Bochner formula and expressible in closed form in terms of degrees and edge weights.
- **Lin–Lu–Yau curvature** (?), which modifies neighborhood measures to better approximate Ollivier curvature on graphs.

Both estimators can be computed in time linear in  $|E|$  and have been shown to correlate well with Ollivier curvature in a variety of graph learning tasks (???). Our empirical results in Section 6 indicate that replacing exact curvature with these proxies has negligible impact on downstream performance, while significantly reducing overhead.

## B. Implementation Details

We describe additional implementation details for Ricci Pruning and Ricci Flow Token Merging, including curvature estimators, clustering algorithms, and hyperparameter choices.

### B.1. Curvature computation

We implement two curvature estimators:

- **Ollivier curvature (approximate).** For each edge  $(i, j)$  we approximate  $W_1(m_i, m_j)$  via a greedy coupling between the top- $k$  neighbors of  $i$  and  $j$ , using edge lengths as ground distances. This yields a good approximation at modest cost when  $k$  is small.
- **Combinatorial curvature.** Following ? and ?, we compute a closed-form curvature proxy based only on degrees and edge weights, with complexity linear in the number of edges.

We use the combinatorial estimator by default and only fall back to Ollivier curvature in diagnostic experiments.

### B.2. Clustering and merging

We cluster candidate tokens in  $\mathcal{F}^{(\ell)}$  using a simple greedy algorithm:

1. Sort tokens by decreasing curvature.
2. For each token  $i$  in this order, if  $i$  is not assigned to any cluster, create a new cluster  $C$  and add all tokens  $j$  such that  $d_{\text{eff}}(i, j) \leq r$ .
3. Stop when the target compression ratio is achieved or all tokens in  $\mathcal{F}^{(\ell)}$  are assigned.

Merged representations are computed as weighted averages of embeddings, keys, and values, with weights proportional to a combination of attention mass and positional encodings.

### B.3. Integration with multi-head attention

We can compute curvature per-head and either:

- *Head-specific pruning:* apply Ricci Pruning separately per head, yielding different compressed KV sets per head; or
- *Shared pruning:* aggregate attention across heads and compute a single curvature profile for all heads.

In our experiments we use shared pruning by default for simplicity and to avoid head-wise fragmentation of tokens.

## C. Additional Experimental Results

This appendix contains extended experimental results, including:

### C.1. Per-dataset breakdown

We provide full results for each benchmark dataset, including:

- Long-document QA datasets with heterogeneous domains (news, scientific articles, legal documents).
- Language modeling datasets spanning narrative text and code.
- Synthetic needle-in-a-haystack setups with varying needle positions.

For each dataset we report EM / F1 or perplexity as appropriate, along with KV memory and throughput.

## C.2. Sensitivity to hyperparameters

We study sensitivity to:

- Curvature thresholds  $\tau_{\text{backbone}}$  and  $\tau_{\text{flat}}$ .
- Cluster radius  $r$  and distance mixture parameter  $\lambda$  in  $d_{\text{eff}}$ .
- Ricci flow step size  $\eta$  and coefficients  $(\gamma_{\text{flat}}, \gamma_{\text{ridge}})$ .

Overall, we find that performance is robust within a reasonable range of hyperparameters, and that curvature-based ranking of tokens is stable across layers and inputs.

## C.3. Case studies

We include qualitative visualizations of:

- Curvature heatmaps over long contexts, highlighting backbone and flat regions.
- Attention patterns before and after Ricci Pruning, showing how important dependencies are preserved.
- Evolution of curvature under Ricci flow across layers.

These case studies further support our central claim: discrete curvature and Ricci flow provide a useful lens for understanding and compressing the internal geometry of long-context LLMs.